Notes C++

EIC95

Commentaires

Les commentaires sur une seule ligne commencent par deux barres obliques (**//**). Les commentaires sur plusieurs lignes commencent par **/*** et se terminent par ***/**.

```
#include <iostream>

using namespace std;

int main()
{
    //azertyuiop
    /*qwerty
    uiop*/
}
```

Flux de sortie

L'objet **cout**, avec l'opérateur **<<**, est utilisé pour imprimer du texte ou des valeur. Pour insérer une nouvelle ligne, vous pouvez utiliser le caractère spécial **\n** ou utiliser le manipulateur **endl**.

```
#include <iostream>

using namespace std;

int main()
{
    cout << "\nHello world" << endl;
}
```

Variables

Pour créer une variable, spécifiez le type et attribuez-lui une valeur et utilisez le mot-clé **const** pour les constantes.

```
#include <iostream>

using namespace std;

int main()
{
    int a = 50;
    int const pi = 3.14;
}
```

- Il ne peut y avoir que des minuscules, des majuscules et des chiffres ;
- Votre nom de variable doit commencer par une lettre ;
- Les espaces sont interdits mais on peut utiliser le caractère "underscore" _ pour séparer des mots. C'est le seul caractère différent des lettres et chiffres autorisé ;
- Les accents ("é", "à", "ê", etc) et les symboles (+, -, *, /, %, =) sont interdits.

Types de données

Type de données	Description	Taille (en octets)
bool	Type booléen représentant vrai (true) ou faux (false)	1
char	Caractère ASCII	1
int	Entier signé	4
float	Nombre à virgule flottante, simple précision	4
double	Nombre à virgule flottante, double précision	8

Saisies utilisateur

cin , associé aux chevrons **>>**, permet de faire entrer des informations dans le programme.

```
#include <iostream>

using namespace std;

int main()
{
    int age;
    cin >> age;
    cout << age;
}
```

Utilisez la fonction **getline()** de la bibliotheque **string** pour lire tout une ligne sans tenir compte des espaces comme le fait cin. Cependant après avoir utilisé cin, si vous passez à lire une chaîne avec getline, utiliser **cin.ignore()** peut être nécessaire pour éviter que getline ne lise accidentellement le retour à la ligne restant dans le tampon.

```cpp
#include <iostream>
#include <string>

int main() {
    std::string message;
    std::cout << "Enter a message: ";
    std::getline(std::cin, message);
    std::cout << "You entered: " << message << std::endl;
    return 0;
}
```

En C++, **cin.ignore()** est utilisé pour ignorer les caractères restants dans le tampon d'entrée après une opération de lecture, comme la lecture d'un seul caractère, afin de préparer le flux d'entrée pour la prochaine opération de lecture.

Opérateurs

Arithmétiques

Opérateur	Description
+	Addition
-	Soustraction
*****	Multiplication
/	Division
%	Modulo (reste de la division)

Raccourcis

+=	Addition et affectation
-=	Soustraction et affectation
***=**	Multiplication et affectation
/=	Division et affectation
%=	Modulo et affectation
++	Incrémentation (postfixe ou préfixe)
--	Décrémentation (postfixe ou préfixe)

Comparaison

==	Égalité
!=	Inégalité
<	Inférieur à
>	Supérieur à
<=	Inférieur ou égal à
>=	Supérieur ou égal à

Logique

Opérateur	Description	Exemple
&&	ET logique	a && b
||	OU logique	a || b
!	NON logique (négation)	!a

Chaînes de caractéres

Les chaînes de caractères en C++ peuvent être manipulées de manière simple et sécurisée en utilisant la classe **std::string** de la bibliothèque standard. Elle offre une variété de méthodes pour manipuler les chaînes de manière flexible et puissante.

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    string name = "Doe";
    string firstName = "John";
    cout << firstName << " " << name;
}
```

size() ou length()	Retourne la longueur de la chaîne.
empty()	Vérifie si la chaîne est vide.
clear()	Efface le contenu de la chaîne.
operator[]	Accède à un caractère à une position spécifiée.
find(sub)	Retourne la position de la première occurrence de sub.
replace(pos, len, str)	Remplace une portion de la chaîne par str.
append(str)	Ajoute str à la fin de la chaîne.

Caractères spéciaux

\n	Nouvelle ligne
\t	Tabulation horizontale
****	Barre oblique inverse
\"	Guillemets doubles
\'	Apostrophe ou guillemets simples
\r	Retour chariot
\b	Retour arrière
\f	Saut de page

Mathématiques

C++ possède de nombreuses fonctions qui vous permettent d'effectuer des tâches mathématiques sur des nombres avec la bibliothéque **cmath**.

Fonction	Description
abs(x)	Retourne la valeur absolue de x.
sqrt(x)	Retourne la racine carrée de x.
pow(base, exp)	Retourne base élevé à la puissance exp.
sin(x)	Retourne le sinus de x (en radians).

cos(x)	Retourne le cosinus de x (en radians).
tan(x)	Retourne la tangente de x (en radians).
log(x)	Retourne le logarithme naturel (base e) de x.
exp(x)	Retourne e élevé à la puissance x.
ceil(x)	Retourne le plus petit entier supérieur ou égal à x.
floor(x)	Retourne le plus grand entier inférieur ou égal à x.

Conditions

if

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    if (20 > 18) {
        cout << "20 is greater than 18";
    }
}
```

if...else

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    int time = 20;
    if (time < 18) {
        cout << "Good day.";
    } else {
        cout << "Good evening.";
    }
}
```

else if

```
#include <iostream>

using namespace std;

int main() {
    int time = 22;
    if (time < 10) {
        cout << "Good morning.";
    } else if (time < 20) {
        cout << "Good day.";
    } else {
        cout << "Good evening.";
    }
}
```

Ternaire

```
#include <iostream>

using namespace std;

int main() {
    int time = 20;
    string result = (time < 18) ? "Good day." : "Good evening.";
    cout << result;
}
```

switch case

```cpp
#include <iostream>

using namespace std;

int main() {
int day = 4;
     switch (day) {
          case 1:
               cout << "Monday";
               break;
          case 2:
               cout << "Tuesday";
               break;
          case 3:
               cout << "Wednesday";
               break;
          case 4:
               cout << "Thursday";
               break;
          case 5:
               cout << "Friday";
               break;
          case 6:
               cout << "Saturday";
               break;
          case 7:
               cout << "Sunday";
               break;
     }
}
```

Boucles

while

```
#include <iostream>

using namespace std;

int main() {
    int i = 0;
    while (i < 5) {
        cout << i << "\n";
        i++;
    }
}
```

do...while

```
#include <iostream>

using namespace std;

int main() {
    int i = 0;
    do {
        cout << i << "\n";
        i++;
    }while (i < 5);
}
```

for

```
#include <iostream>

using namespace std;

int main() {
    for (int i = 0; i < 5; i++) {
        cout << i << "\n";
    }
}
```

Il existe également une boucle **for-each** (introduite dans la version C++ 11 (2011), qui est utilisée exclusivement pour parcourir les éléments d'un tableau (ou d'autres ensembles de données)

```
#include <iostream>

using namespace std;

int main() {
    int myNumbers[5] = {10, 20, 30, 40, 50};
    for (int i : myNumbers) {
        cout << i << "\n";
    }
}
```

L'instruction **break** peut être utilisée pour sortir d'une boucle et L'instruction **continue** interrompt une itération (dans la boucle), si une condition spécifiée se produit, et continue avec l'itération suivante dans la boucle.

Tableaux

Pour déclarer un tableau, définissez le type de variable, spécifiez le nom du tableau suivi de crochets et spécifiez le nombre d'éléments qu'il doit stocker et pour déclarer un tableau multidimensionnel, définissez le type de variable, spécifiez le nom du tableau suivi de crochets qui spécifient le nombre d'éléments du tableau principal, suivi d'un autre ensemble de crochets qui indique le nombre d'éléments des sous-tableaux. Pour obtenir la taille d'un tableau, vous pouvez utiliser opérateur **sizeof().**

```cpp
#include <iostream>

using namespace std;

int main() {
    int myNumbers[5] = {10, 20, 30, 40, 50};
    int two_dimension[2][2] = {{1,2},{3,4}};
}
```

Un **vecteur** (**std::vector**) en C++ est un conteneur de données dynamique qui permet de stocker une collection d'éléments de même type de manière flexible. Contrairement aux tableaux statiques comme std::array, la taille d'un vecteur peut être modifiée dynamiquement pendant l'exécution du programme. Pour utiliser des tableaux dynamiques, il faut inclure la bibliothèque **<vector>**, qui fournit aussi des fonctions pour manipuler les vecteurs.

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector <int> number(3);
}
```

Méthode	Description
push_back(value)	Ajoute un élément à la fin du vecteur.
pop_back()	Supprime l'élément à la fin du vecteur.
at(index)	Accède à l'élément à l'indice spécifié avec vérification des limites.
size()	Retourne le nombre d'éléments dans le vecteur.
std::sort(begin, end)	Trie les éléments du vecteur en utilisant l'opérateur <.

Structures

Pour créer une structure, utilisez le mot-clé **struct** et déclarez chacun de ses membres entre accolades. Après la déclaration, précisez le nom de la variable de structure. Pour accéder aux membres d'une structure, utilisez la syntaxe à points (**.**).

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  struct {
    int myNum;
    string myString;
  } myStructure;

  myStructure.myNum = 1;
  myStructure.myString = "Hello World!";

  cout << myStructure.myNum << "\n";
  cout << myStructure.myString << "\n";
  return 0;
}
```

Énumérations

Pour créer une énumération, utilisez le mot-clé **enum**, suivi du nom de l'énumération, et séparez les éléments de l'énumération par une virgule. Pour accéder à l'énumération, vous devez créer une variable de celle-ci. À l'intérieur de la méthode main(), spécifiez le mot-clé enum, suivi du nom de l'énumération. Par défaut, le premier élément a la valeur 0, le deuxième a la valeur 1, etc. Pour donner plus de sens aux valeurs, vous pouvez facilement les modifier, Mais notez que si vous attribuez une valeur à un élément spécifique, les éléments suivants mettront à jour leurs numéros en conséquence.

```cpp
#include <iostream>
using namespace std;

enum Level {
  LOW = 10 ,
  MEDIUM, // 11
  HIGH // 12
};

int main() {
     enum Level myVar = MEDIUM;
     cout << myVar;
     return 0;
}
```

Références

Une référence est un alias pour une autre variable. Plutôt que de travailler directement avec la variable d'origine, une référence permet d'y accéder indirectement. Pour déclarer une référence, on utilise le symbole **&** Une fois initialisée, une référence ne peut pas être réassignée pour référencer une autre variable. Une référence doit être initialisée au moment de sa déclaration. Passée a une fonction, elle permet de modifier les arguments d'une fonction ou d'éviter la copie coûteuse de grands objets.

```cpp
#include <iostream>
using namespace std;

void increment(int& n) {
    n++;
}

int main() {
    int x = 10;
    int& ref = x;
    ref = 15; // x = 15
    increment(x);
    cout << x ; // 16
    return 0;
}
```

Utilisées avec le mot clé **const**, Elle evite la copie tout en protégeant les données d'origine de modifications.

```
#include <iostream>
using namespace std;

void print(const int& nbr) {
    cout << nbr << endl;
}

int main() {
    int x = 10;
    print(x);
    return 0;
}
```

Pointeurs

Un pointeur est une variable qui stocke l'adresse mémoire d'une autre variable. Pour déclarer un pointeur, on utilise le symbole *****. Les pointeurs doivent être du même type que la variable qu'ils pointent. Un pointeur peut être nul, signifiant qu'il ne pointe vers aucune adresse valide. On utilise généralement la constante **nullptr** pour initialiser un pointeur nul. Utilisez l'opérateur * pour accéder ou modifier la valeur pointée par le pointeur. Il permettent de modifier les arguments d'une fonction ou de gérer des tableaux dynamiques.

```cpp
#include <iostream>
using namespace std;

void print(int* nbr) {
    cout << *nbr << endl;
}

int main() {
    int x = 10;
    int* ptr = &x;
    print(ptr);
    return 0;
}
```

Fonctions

Une fonction en C++ est définie par un type de retour, un nom, une liste de paramètres (facultative), et un corps de fonction. Pour appeler une fonction, on utilise son nom suivi de parenthèses contenant les arguments. Les fonctions peuvent retourner une valeur à l'aide de l'instruction **return**. Une fonction qui ne retourne pas de valeur a un type de retour **void**. C++ permet la **surcharge de fonctions**, c'est-à-dire la définition de plusieurs fonctions ayant le même nom mais des listes de paramètres différentes.

```cpp
#include <iostream>
using namespace std;

int add(int nbr , int nbr_2) {
    return nbr + nbr_2;
}

int main() {
    int x = 10;
    int y = 20;
    cout << add(x,y);
    return 0;
}
```

Classes et objets

Pour créer une classe, utilisez le mot-clé **class** Pour créer un objet, spécifiez le nom de la classe, suivi du nom de l'objet et accédez aux attributs de classe avec la syntaxe à points (.) sur l'objet.

```
#include <iostream>
#include <string>
using namespace std;

class MyClass {
  public:
    int myNum;
    string myString;
};

int main() {
  MyClass myObj;

  myObj.myNum = 15;
  myObj.myString = "Some text";

  cout << myObj.myNum << "\n";
  cout << myObj.myString;
  return 0;
}
```

Il existe deux manières de définir des fonctions appartenant à une classe :

- Définition de la classe intérieure
- Définition de la classe extérieure

Pour définir une fonction en dehors de la définition de classe, vous devez la déclarer à l'intérieur de la classe, puis la définir en dehors de la classe. Cela se fait en spécifiant le nom de la classe, suivi de l'opérateur **::** de résolution de portée, suivi du nom de la fonction

Remarque : vous accédez aux méthodes de la même manière que vous accédez aux attributs ; en créant un objet de la classe et en utilisant la syntaxe à points (**.**).

```cpp
#include <iostream>
#include <string>
using namespace std;


class MyClass {
  public:
    void myMethod() {
      cout << "Hello ";
    }
    void mySecondMethod();
};

void MyClass::mySecondMethod(){
     cout << "John Doe";
}

int main() {
  MyClass myObj;
  myObj.myMethod();
  myObj.mySecondMethod();
  return 0;
}
```

Pour créer un constructeur, utilisez le même nom que la classe, suivi de parenthèses **()**. Les constructeurs peuvent également prendre des paramètres (tout comme les fonctions classiques), ce qui peut être utile pour définir les valeurs initiales des attributs. out comme les fonctions, les constructeurs peuvent également être définis en dehors de la classe. Tout d'abord, déclarez le constructeur à l'intérieur de la classe, puis définissez-le en dehors de la classe en spécifiant le nom de la classe, suivi de l'opérateur **::** de résolution de portée, suivi du nom du constructeur (qui est le même que la classe).

```
#include <iostream>
using namespace std;

class Car {
  public:
    string brand;
    string model;
    int year;
    Car(string x, string y, int z) {
      brand = x;
      model = y;
      year = z;
    }
};

int main() {

  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
  return 0;
}
```

Spécificateurs d'accès

Mot-clé	Description	Accessibilité
public	Les membres déclarés comme publics sont accessibles depuis l'extérieur de la classe.	Accessibles et modifiables depuis l'extérieur.
private	Les membres déclarés comme privés ne sont accessibles que depuis l'intérieur de la classe elle-même.	Non accessibles depuis l'extérieur.

protected	Les membres déclarés comme protégés sont accessibles dans les classes dérivées mais pas depuis l'extérieur.	Accessibles dans les classes dérivées.
static	Déclare des membres de classe et des méthodes qui appartiennent à la classe elle-même plutôt qu'aux instances spécifiques.	Modifiables sans créer d'instance.

Encapsulation

L' **encapsulation** a pour but de s'assurer que les données « sensibles » sont cachées aux utilisateurs. Pour y parvenir, vous devez déclarer les variables/attributs de classe comme private(non accessibles depuis l'extérieur de la classe). Si vous souhaitez que d'autres personnes lisent ou modifient la valeur d'un membre privé, vous pouvez fournir des méthodes **get** et **set** publiques .

```cpp
#include <iostream>
using namespace std;

class Employee {
  private:
    int salary;

  public:
    void setSalary(int s) {
      salary = s;
    }
    int getSalary() {
      return salary;
    }
};

int main() {
  Employee myObj;
  myObj.setSalary(50000);
  cout << myObj.getSalary();
  return 0;
}
```

Héritage

Pour hériter d'une classe, utilisez le symbole (:). Une classe peut hériter de plusieurs classes, il suffit de les séparer par des virgules

```
#include <iostream>
using namespace std;

class Vehicle {
  public:
    string brand = "Ford";
    void honk() {
      cout << "Tuut, tuut! \n" ;
    }
};

class Car: public Vehicle {
  public:
    string model = "Mustang";
};

int main() {
  Car myCar;
  myCar.honk();
  cout << myCar.brand + " " + myCar.model;
  return 0;
}
```

Type d'héritage	Membres publics dans la classe dérivée	Membres protégés dans la classe dérivée	Membres privés dans la classe dérivée
public	Restent publics	Restent protégés	Restent privés
protected	Deviennent protégés	Restent protégés	Restent privés
private	Deviennent privés	Deviennent privés	Restent privés

Polymorphisme

Le **polymorphisme** signifie « plusieurs formes » et se produit lorsque de nombreuses classes sont liées entre elles par héritage. Il permet aux classes dérivées d'hériter des attributs et des méthodes d'une classe de base et d'utiliser ces méthodes pour effectuer différentes tâches selon la classe dérivée. Ainsi, une même action peut être réalisée de manières différentes en fonction de l'objet sur lequel elle est appelée.

```cpp
#include <iostream>
using namespace std;

class Animal {
  public:
    void animalSound() {
      cout << "The animal makes a sound \n";
    }
};
class Pig : public Animal {
  public:
    void animalSound() {
      cout << "The pig says: wee wee \n";
    }
};
class Dog : public Animal {
  public:
    void animalSound() {
      cout << "The dog says: bow wow \n";
    }
};

int main() {
  Animal myAnimal;
  Pig myPig;
  Dog myDog;
  myAnimal.animalSound();
  myPig.animalSound();
  myDog.animalSound();
  return 0;
}
```

Gestion des fichiers

La bibliothèque **fstream** nous permet de travailler avec des fichiers. Pour utiliser la bibliothèque fstream, incluez à la fois le fichier standard **<iostream>** et le fichier d'en-tête **<fstream>**. La bibliothèque comprend trois classes utilisées pour créer, écrire ou lire des fichiers.

ofstream	Crée et écrit dans des fichiers
ifstream	Lit à partir de fichiers
fstream	Une combinaison de ofstream et ifstream : crée, lit et écrit dans des fichiers

Pour créer un fichier, utilisez la classe ofstream ou fstream et spécifiez le nom du fichier. Pour écrire dans le fichier, utilisez l'opérateur d'insertion (**<<**). Cela est considéré comme une bonne pratique de fermer le fichier et pour nettoyer l'espace mémoire inutile.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
  ofstream MyFile("filename.txt");

  MyFile << "Files can be tricky, but it is fun enough!";

  MyFile.close();
}
```

Pour lire un fichier, utilisez soit la classe ifstream ou fstream , puis le nom du fichier. Notez que nous utilisons également une boucle while avec la fonction getline() (qui appartient à la classe ifstream) pour lire le fichier ligne par ligne et pour imprimer le contenu du fichier.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    string myText;
    ifstream MyReadFile("filename.txt");
    while (getline (MyReadFile, myText)) {
        cout << myText;
    }
    MyReadFile.close();
}
```

Exceptions

La gestion des exceptions en C++ utilise trois mots-clés principaux : **try**, **throw**, et **catch**.

- try : Ce mot-clé introduit un bloc de code dans lequel des exceptions peuvent se produire. C'est là que vous placez le code susceptible de générer des erreurs.
- throw : Lorsqu'une condition d'erreur est rencontrée dans le bloc try, le mot-clé throw est utilisé pour lancer une exception. Cela signale une erreur et peut être utilisé pour créer des exceptions personnalisées.

- catch : Le mot-clé catch est utilisé pour attraper (ou capturer) une exception qui a été lancée (throw) dans le bloc try. Vous pouvez définir plusieurs blocs catch pour gérer différents types d'exceptions ou scénarios d'erreur.

```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    try {
        int age = 15;
        if (age >= 18) {
            cout << "Access granted - you are old enough.";
        } else {
            throw (age);
        }
    }catch (int myNum) {
        cout << "Access denied - You must be at least 18 years old.\n";
        cout << "Age is: " << myNum;
    }
}
```

Si vous ne connaissez pas le type throw utilisé dans le bloc try, vous pouvez utiliser la syntaxe « trois points » (**...**) à l'intérieur du bloc catch, qui gérera tout type d'exception.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    try {
        int age = 15;
        if (age >= 18) {
            cout << "Access granted - you are old enough.";
        } else {
            throw 505;
        }
    }catch (...) {
        cout << "Access denied - You must be at least 18 years old.\n";
    }
}
```

Programmation modulaire

La **programmation modulaire** est une technique qui consiste à diviser un programme en sous-programmes ou modules pour accomplir des tâches spécifiques. Chaque module est conçu pour réaliser une partie de la fonctionnalité globale et est conçu de manière à être réutilisable. Les fonctions sont déclarées dans des fichiers d'en-tête (**.hpp**) et définies dans des fichiers sources (**.cpp**). Les fichiers d'en-tête sont inclus dans les fichiers sources via la directive **#include**.

Un **prototype de fonction** est une déclaration de la fonction qui informe le compilateur de la manière dont la fonction est utilisée. Il comprend le type de retour de la fonction, le nom de la fonction, les types d’arguments qu’elle prend et un point-virgule. Supposons que vous ayez une fonction add qui additionne deux nombres. Vous pouvez mettre le prototype de la fonction dans un fichier d’en-tête add.hpp, la définition de la fonction dans un fichier source add.cpp, et ensuite inclureadd.hpp dans tout autre fichier qui utilise la fonction add.

add.hpp

```
#ifndef ADD_H
#define ADD_H

int add(int a, int b);

#endif
```

add.cpp

```cpp
#include "add.hpp"

int add(int a , int b){
    return a + b;
}
```

demo.cpp

```cpp
#include <iostream>
#include "add.hpp"
using namespace std;

int main() {
    int x = 10;
    int y = 20;
    cout << add(x,y);
}
```

Pour la compilation, nous ecrirons **g++ -o demo.exe demo.cpp add.cpp**. Les lignes **ifndef**, **define** et **endif** sont là pour empêcher le compilateur d'inclure plusieurs fois ce fichier.

FIN